Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky

SalonCRM – CRM systém pre kozmetický salón

Semestrálny projekt

Bc. Martin Saxa
Bc. Jakub Spišák
Bc. Roman Šafranko
Bc. Samuel Šarkan

Košice 2026

Obsah

1 Zadanie a analýza **1**

1.1 Zadanie 1

1.2 Rozdelenie úloh v tíme 1

1.3 Analýza domény 2

1.4 Predikcia odchodu zákazníka 3

2 Architektúra **4**

2.1 Komponenty 5

2.2 Komunikácia medzi službami 5

2.3 Autentifikácia a sessions 6

2.4 ML pipeline 6

3 Nasadenie **8**

3.1 Kontajnery 8

3.2 Nasadenie na Azure 9

4 Záver **12**

1 Zadanie a analýza

1.1 Zadanie

Cieľom projektu je vytvoriť vnútropodnikový informačný systém pre kozmetický salón. Systém má podobu CRM (*Customer Relationship Management*) a slúži na správu zákazníkov, objednávok (termínov) a platieb. Okrem bežnej evidencie má salónu pomôcť udržať si zákazníkov: pomocou strojového učenia predikuje, ktorí zákazníci pravdepodobne prestanú chodiť, aby ich mohol personál včas osloviť.

Zo zadania predmetu vyplývajú tieto technické požiadavky:

- podnikový informačný systém postavený na prepojených aplikačných webových službách,
- prihlasovanie a odhlasovanie, aspoň dve používateľské roly a práca so sessions,
- hlavný backend, ktorý obsluhuje aspoň dve obrazovky,
- druhý backend v inom programovacom jazyku (nemusí mať vlastné používateľské rozhranie),
- frontend, ktorý spája dáta z oboch backendov,
- dve samostatné SQL databázy,
- komponent strojového učenia,
- procesný model v notácii BPMN,
- nasadenie pomocou kontajnerov.

1.2 Rozdelenie úloh v tíme

Projekt riešil štvorčlenný tím. Rozdelenie rolí uvádza tabuľka 1.1.

Tabuľka 1.1: Roly členov tímu

Rola	Člen tímu	Zodpovednosť
AI expert	Roman Šafranko	ML služba, príznaky, tréning a vyhodnotenie modelu
Doménový expert	Martin Saxa	požiadavky salónu, analýza domény, procesy (BPMN)
Softvérový architekt	Samuel Šarkan	architektúra, rozhrania služieb, nasadenie
Softvérový inžinier	Jakub Spišák	implementácia backendu a frontendu

1.3 Analýza domény

Kozmetický salón ponúka služby rozdelené do kategórií (ošetrenie pleti, nechty, obočie a mihalnice, depilácia, masáže, líčenie). Každá služba má dĺžku trvania a cenu a vykonávajú ju iba niektorí pracovníci. Salón má pevné otváracie hodiny a termíny sa plánujú v 15-minútovej mriežke.

V systéme vystupujú tri skupiny používateľov:

Zákazník nemá účet. Rezervuje si termín cez verejný formulár: vyberie službu, pracovníka (alebo „ktokoľvek“), voľný čas a zadá kontaktné údaje. Identifikuje sa e-mailom; ak zákazník s rovnakým e-mailom už existuje, termín sa priradí k nemu.

Pracovník (**rola `User`**) napríklad kozmetička. Vidí svoj kalendár, spravuje iba svoje termíny a iba zákazníkov, ktorých obsluhoval. Termín môže dokončiť (a zaevidovať platbu), zrušiť, označiť ako nedostavenie sa alebo preplánovať. Telefonické objednávky zapisuje za zákazníka.

Administrátor (**rola `Admin`**) vidí všetky dáta, spravuje používateľov, služby a cenník, sleduje štatistiky (tržby, návštevy, vyťaženosť pracovníkov) a pracuje s výsledkami predikcie odchodu zákazníkov.

Životný cyklus termínu je jednoduchý stavový automat. Nový termín je v stave *naplánovaný*. Odtiaľ môže prejsť do stavu *dokončený* (spolu so zápisom platby), *zrušený* alebo *neprišiel*. Všetky tri sú konečné. Dôležitým obchodným pravidlom je, že pracovník nesmie mať dva prekrývajúce sa termíny. Kontrola iba v apliká-

cii nestačí, pretože dvaja zákazníci môžu ten istý čas rezervovať súčasne. Preto prekrývaniu bráni aj obmedzenie priamo v databáze.

1.4 Predikcia odchodu zákazníka

Pre salón je lacnejšie udržať si existujúceho zákazníka ako získať nového. Systém preto pre každého zákazníka s aspoň dvoma dokončenými návštevami odhaduje pravdepodobnosť, že v nasledujúcich 90 dňoch nepríde (tzv. *churn*). Výsledok má podobu pravdepodobnosti, úrovne rizika (nízke, stredné, vysoké) a čitateľného zdôvodnenia, napríklad: „Posledná návšteva pred 64 dňami, obvykle chodí každých 21 dní. Nemá naplánovaný ďalší termín.“

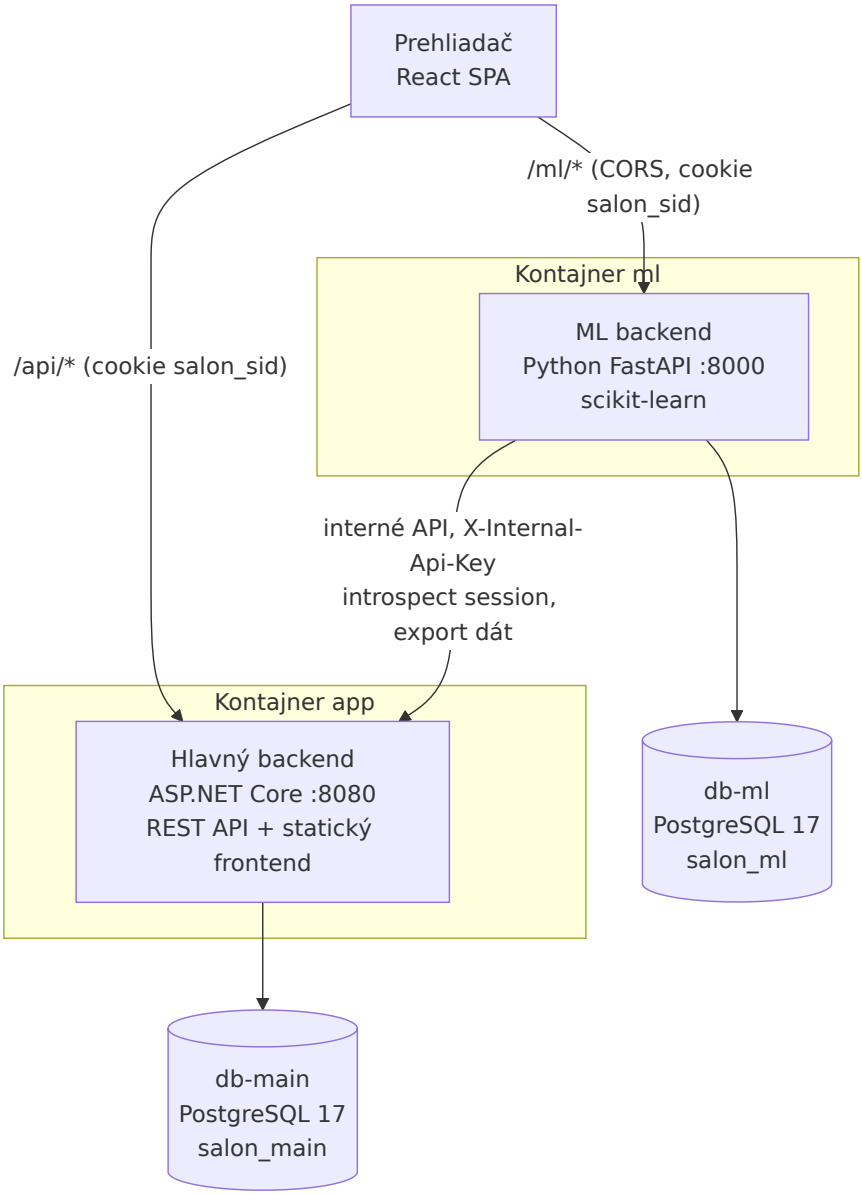
Model sa učí z histórie termínov. Príznaky opisujú správanie zákazníka k zvolenému dátumu: počet dní od poslednej návštevy, priemerný interval medzi návštevami a ich pomer, počet návštev za posledných 90 a 180 dní, útrata, podiel zrušených termínov, existencia budúceho termínu a najčastejšia kategória služby. Pri ich výpočte sa používajú iba údaje, ktoré boli k danému dátumu známe, aby model „nevidel do budúcnosti“ (*data leakage*).

Trénuje sa niekoľko klasifikátorov z knižnice scikit-learn (logistická regresia, náhodný les a gradient boosting). Vyhodnocujú sa na časovo oddelenej validačnej množine a porovnávajú s jednoduchým pravidlom (zákazník nechodí viac ako dvojnásobok svojho obvyklého intervalu). Vyberie sa model s najvyššou hodnotou ROC-AUC. Na základe predikcie môže administrátor zaznamenať *retenčnú akciu*, teda kontakt so zákazníkom telefonicky, e-mailom alebo osobne.

Keďže salón nemá reálne historické dáta, systém pracuje so syntetickými dátami. Tie modelujú rôzne typy zákazníkov (verní, nepravidelní, odchádzajúci, jednorazoví) aj so zámerným šumom. Dosiahnuté metriky preto preukazujú funkčnosť celého postupu, nie presnosť na reálnych dátach.

2 Architektúra

Systém tvoria štyri hlavné časti: webový frontend, hlavný backend, ML backend a dve databázy. Ich prepojenie zobrazuje obrázok 2.1.



Obrázok 2.1: Architektúra systému SalonCRM

2.1 Komponenty

Frontend je jednostránková aplikácia (SPA) napísaná v jazyku TypeScript s knižnicou React a nástrojom Vite. Na načítanie dát používa TanStack Query, formuláre validuje knižnicou zod, kalendár termínov zobrazuje cez FullCalendar a grafy cez Recharts. Obsahuje dvoch API klientov: jeden pre hlavný backend (`/api/*`) a druhý pre ML backend (`/ml/*`). Viaceré obrazovky kombinujú dáta z oboch. Napríklad kalendár a zoznam zákazníkov dopĺňajú k zákazníkom farebný odznak rizika odchodu a ML dashboard dopĺňa k predikciám mená zákazníkov z hlavného backendu.

Hlavný backend je REST API v ASP.NET Core (.NET 10) s prístupom do databázy cez Entity Framework Core. Je usporiadaný do vertikálnych rezov podľa oblastí (autentifikácia, verejná rezervácia, termíny, zákazníci, používatelia, služby, štatistiky). Obsluhuje všetky obrazovky okrem ML dashboardu a v produkcii servíruje aj zostavený frontend.

ML backend je REST API v jazyku Python s frameworkom FastAPI. Nemá vlastné používateľské rozhranie. Na prácu s dátami používa pandas, na strojové učenie scikit-learn a natrénované modely ukladá cez joblib. Zabezpečuje synchronizáciu dát, tréning, predikciu a sprístupnenie výsledkov.

Databázy. Každý backend má vlastnú inštanciu PostgreSQL 17. Databáza `salon_main` obsahuje používateľov, sessions, služby, zákazníkov, termíny, platby a retenčné akcie. Databáza `salon_ml` obsahuje kópiu pseudonymizovaných dát na tréning, verzie modelov s metrikami a históriu predikcií. Žiadny backend nečíta databázu toho druhého. Jediným prepojením je HTTP komunikácia.

2.2 Komunikácia medzi službami

Frontend komunikuje s oboma backendmi priamo. ML backend volá interné API hlavného backendu, ktoré je chránené zdieľaným kľúčom v hlavičke `X-Internal-Api-Key`. Interné API má dva endpointy:

- `POST/api/internal/sessions/introspect` overí session prihláseného používateľa a vráti jeho rolu,
- `GET/api/internal/ml/export` vráti zákazníkov a termíny na tréning modelu.

Export obsahuje iba pseudonymné identifikátory a údaje o termínoch (čas, stav, kategória služby, cena). Mená, e-maily, telefóny a poznámky do ML služby neprechádzajú, čím sa dodržiava princíp minimalizácie dát.

2.3 Autentifikácia a sessions

Prihlásenie zabezpečuje hlavný backend. Po overení hesla vygeneruje náhodný token, ktorý odošle prehliadaču v cookie `salon_sid` s príznakmi `HttpOnly` a `SameSite=Lax`. V tabuľke `sessions` sa uloží iba hash tokenu. Session platí najviac 12 hodín a po dvoch hodinách nečinnosti vyprší. Odhlásenie session zneplatní a administrátor vidí zoznam aktívnych sessions a môže ich rušiť.

ML backend vlastné prihlasovanie nemá. Pri chránenom požiadavku prečíta tú istú cookie a overí ju cez introspekciu v hlavnom backende. Výsledok si na krátky čas uloží do cache. Cookie sa na ML backend dostane vďaka tomu, že oba backendy patria k rovnakému webu (*site*): lokálne bežia na rovnakom hostiteľovi s iným portom, v produkcii na subdoménach jednej domény. ML backend povoľuje CORS s prihlasovacími údajmi iba pre konkrétne originy.

Autorizácia je explicitná na každom endpointe. Obmedzenie pracovníka iba na jeho vlastné termíny a zákazníkov je súčasťou databázového dotazu, nie iba používateľského rozhrania. Záznam, ktorý pracovníkovi nepatrí, sa správa, akoby neexistoval (HTTP 404).

2.4 ML pipeline

Administrátor spúšťa pipeline tlačidlom „Prepočítať predikcie“ v ML dashboarde. Beh má tieto kroky:

1. stiahnutie exportu z hlavného backendu a jeho uloženie do `salon_ml`,
2. vytvorenie trénovacích príkladov z historických snapshotov (vždy s celým 90-dňovým horizontom v minulosti, aby bol známy skutočný výsledok),
3. tréning kandidátov a ich vyhodnotenie na najnovšom snapshote voči jednoduchému pravidlu,
4. výber najlepšieho modelu, výpočet dôležitosti príznakov, pretrénovanie na všetkých dátach a uloženie novej verzie modelu,
5. predikcia pre všetkých zákazníkov s dostatočnou históriou a uloženie výsledkov s vysvetlením.

Predikcie sa nemažú, takže je možné sledovať vývoj rizika konkrétneho zákazníka v čase.

3 Nasadenie

3.1 Kontajnery

Celý systém je zabalený do kontajnerov. Na lokálny vývoj a demonštráciu sa spúšťa pomocou Docker Compose. Základ tvoria štyri kontajnery (tabuľka 3.1).

Tabuľka 3.1: Kontajnery systému

Kontajner	Port	Obsah
`db-main`	5432	PostgreSQL 17, databáza `salon_main`
`db-ml`	5433	PostgreSQL 17, databáza `salon_ml`
`app`	8080	ASP.NET Core API spolu so zostaveným frontendom
`ml`	8000	FastAPI a ML pipeline

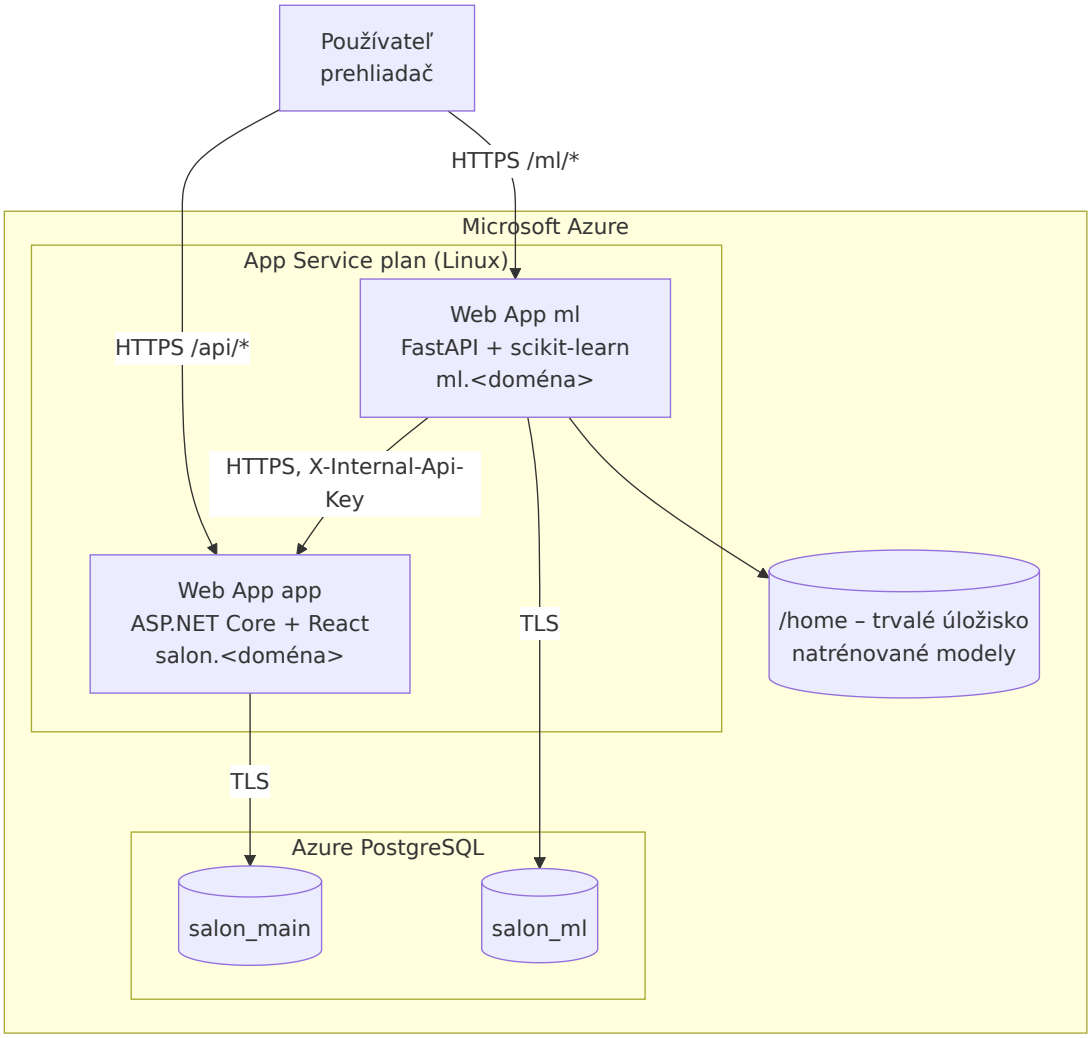
Obraz `app` sa zostavuje viacstupňovo: najprv sa v Node.js zostaví frontend, potom sa v .NET SDK skompiluje backend a nakoniec sa oba výstupy skopírujú do runtime obrazu ASP.NET Core. Hlavný backend tak servíruje statické súbory frontendu a všetky cesty mimo `/api` presmeruje na `index.html`, kým neznáma API cesta vráti chybu 404 vo formáte JSON. Pri štarte kontajnera sa automaticky aplikujú databázové migrácie a naplnia sa ukážkové dáta. Kontajner `ml` pri štarte aplikuje migrácie nástrojom Alembic a spustí server uvicorn.

Databázy majú zdravotné kontroly (`pg_isready`) a backendy štartujú až po tom, čo je ich databáza pripravená. Dáta oboch databáz a natrénované modely sú uložené v pomenovaných zväzkoch, takže prežijú reštart aj výmenu kontajnerov. Konfigurácia a tajomstvá (heslá databáz, interný API kľúč, povolené originy pre CORS, adresa ML služby) sa odovzdávajú výhradne cez premenné prostredia zo súboru `.env`.

Lokálne sa celý systém spustí príkazom `docker compose up --build` a je dostupný na adrese `http://localhost:8080`.

3.2 Nasadenie na Azure

V produkcii sa namiesto Docker Compose použijú spravované služby platformy Microsoft Azure (obrázok 3.1). Oba backendy bežia ako kontajnery v službe Azure App Service (Web App for Containers) a databázy poskytuje služba Azure Database for PostgreSQL. Azure sa tak stará o prevádzku servera, aktualizácie, HTTPS certifikáty aj zálohovanie databáz.



Obrázok 3.1: Nasadenie v Azure

Aplikácie

V jednom App Service pláne s Linuxom bežia dve webové aplikácie. Každá spúšťa ten istý Docker obraz, ktorý sa používa lokálne:

Web App app hlavný backend spolu so zostaveným frontendom,

Web App `ml` ML backend.

App Service zabezpečuje HTTPS, preto sa cookie `salon_sid` posiela s príznakom `Secure`. Obe aplikácie dostanú vlastnú doménu ako subdomény jednej registrovanej domény (napr. `salon.<doména>` a `ml.<doména>`). Predvolené adresy `*.azurewebsites.net` sa nepoužívajú, pretože prehliadač ich považuje za rôzne weby (*sites*). Cookie so `SameSite=Lax` by sa potom na ML backend neposlala. Pod spoločnou doménou sa cookie posiela obom backendom rovnako ako pri lokálnom vývoji.

ML backend volá interné API hlavného backendu cez jeho HTTPS adresu, chránené kľúčom v hlavičke `X-Internal-Api-Key`. Natrénované modely sa ukladajú do trvalého úložiska App Service (adresár `/home`, premenná `MODEL_DIR`), takže prežijú reštart aj nasadenie novej verzie.

Databázy

Kontajnery `db-main` a `db-ml` v produkcii nahrádza Azure Database for PostgreSQL – Flexible Server s verziou PostgreSQL 17. Na serveri sú dve samostatné databázy, `salon_main` a `salon_ml`. Každý backend má vlastného databázového používateľa s právami iba na svoju databázu, takže oddelenie dát zostáva zachované. Spojenie je šifrované (TLS) a firewall servera povoľuje prístup iba zo služieb Azure. Rozšírenie `btree_gist`, ktoré potrebuje obmedzenie proti prekrývaniu termínov, treba na serveri povoliť v parametri `azure.extensions`.

Konfigurácia a postup nasadenia

Konfigurácia sa nastavuje v *Application settings* webových aplikácií. Tie App Service odovzdá kontajneru ako premenné prostredia, rovnako ako súbor `.env` lokálne. Ide najmä o reťazce pripojenia k databázam, interný API kľúč, adresu hlavného backendu pre ML službu, povolené originy CORS a príznak `Secure` pre cookie. Adresa ML služby sa zapisuje do frontendu pri zostavení obrazu, preto sa obraz `app` zostavuje už s produkčnou adresou.

Postup nasadenia:

1. vytvorenie skupiny prostriedkov, PostgreSQL servera s dvoma databázami a dvoma používateľmi a povolenie rozšírenia `btree_gist`,
2. vytvorenie App Service plánu a dvoch webových aplikácií pre kontajnery,
3. nastavenie vlastnej domény a certifikátov pre obe aplikácie,

4. nastavenie *Application settings* (tajomstvá iba tu, nie v repozitári),
5. zostavenie Docker obrazov v GitHub Actions a ich nahratie do registra kontajnerov,
6. nasadenie obrazov do webových aplikácií; migrácie databáz sa aplikujú automaticky pri štarte kontajnerov.

Kontinuálna integrácia (GitHub Actions) pri každej zmene zostaví a otestuje všetky časti systému. Rovnaký workflow sa dá rozšíriť o automatické nasadenie nových obrazov po úspešných testoch.

4 Záver

V rámci projektu sme navrhli a implementovali informačný systém SalonCRM pre kozmetický salón. Zákazníci si v ňom môžu bez registrácie online rezervovať termín, pracovníci spravujú svoj kalendár, termíny a platby a administrátor spravuje používateľov, služby a cenník a sleduje štatistiky salónu.

Systém spĺňa technické požiadavky zadania:

- dve prepojené webové služby v rôznych jazykoch: hlavný backend v ASP.NET Core a ML backend v Python FastAPI, ktoré komunikujú cez interné REST API,
- prihlasovanie s rolami `Admin` a `User` a server-side sessions uloženými v databáze,
- frontend v Reacte, ktorý volá oba backendy a na viacerých obrazovkách kombinuje ich dáta,
- dve samostatné databázy PostgreSQL, ku ktorým pristupuje vždy iba „ich“ backend,
- predikcia odchodu zákazníka pomocou strojového učenia s porovnaním viacerých modelov a vysvetlením výsledku,
- kontajnerizácia: lokálne Docker Compose so štyrmi kontajnermi, v produkcii Azure App Service a Azure Database for PostgreSQL.

Prínosom pre salón je predovšetkým ML časť: personál nemusí ručne sledovať, kto prestal chodiť, ale dostane zoradený zoznam rizikových zákazníkov aj s dôvodom a môže ich cielene osloviť. Uskutočnené kontakty sa evidujú ako retenčné akcie.

Systém má aj vedomé obmedzenia. E-mail zákazníka sa pri rezervácii neoveruje, model je natrénovaný na syntetických dátach a produkčné nasadenie v Azure si vyžaduje vlastnú doménu. Na projekt možno nadviazať e-mailovými

pripomienkami termínov, zrušením rezervácie zákazníkom cez odkaz, hodnotením návštev (ktoré by mohlo slúžiť aj ako ďalší príznak modelu), a automatickým nasadzovaním po úspešnom prechode CI.